TinyFed6G: Federated Learning with TinyML for Resource-Constrained Intelligence in 6G Edge Networks

Vibha Jain*, Aditya Gupta*, Prabal Verma†, Sukhpal Singh Gill‡
*Thapar Institute of Engineering and Technology, Patiala, INDIA Email: vibha.jain@thapar.edu; aditya.gupta@thapar.edu
†National Institute of Technology, Srinagar, INDIA Email: prabalverma357@gmail.com
‡School of Electronic Engineering and Computer Science, Queen Mary University of London, London, UK Email: s.s.gill@qmul.ac.uk

***Abstract*—The rising prevalence of ultra-low-power microcontrollers in intelligent edge devices has created a demand for collaborative learning on devices. Federated Learning (FL) platforms are designed for edge devices; however, their functionality may be limited due to edge device constraints on memory, energy, and communication bandwidth, particularly in non-IID (Non-Independent and Identically Distributed) data distributions and heterogeneous hardware. To overcome these issues, this paper proposes TinyFed6G, a communication-efficient hierarchical FL framework that is 6G-aligned with TinyML (Tiny Machine Learning) clients. The architecture supports a dual-mode execution that enables the device to dynamically switch between training mode and inference mode based on real-time profiling of the device. Enhanced communication efficiency is achieved through semantic-aware compression, which filters quantized model updates based on a cosine similarity threshold. Personalized learning is accomplished by selectively fine-tuning the head layer of the model. The performance of the framework is evaluated using distributions. Experimental results demonstrate that TinyFed6G achieves a final accuracy of 86.2%, a 6.5% personalization gain.**

***Index Terms*—TinyML, Federated Learning, 6G Networks, On-Device Training, Edge Intelligence, IoT**

I. Introduction

The proliferation of the Internet of Things (IoT) is radically transforming modern computing paradigms by distributing intelligence closer to the physical world. The number of connected devices was predicted to be approximately 9 billion in 2019 and 15 billion in 2023, and it is expected to reach 29 billion by the end of 2030, spanning applications such as smart healthcare, industrial automation, environmental monitoring, and urban mobility [1]. These devices typically operate in constrained settings, powered by ultra-low-power microcontrollers with limited Static Random Access Memory (SRAM) ($\leq$ 256 KB), minimal computational throughput, and limited energy. While ideal for basic sensing and communication, such hardware is inadequate for conventional Machine Learning (ML) tasks that demand high memory, compute capacity, and centralized data aggregation.

To deal with the computing ability problem of resource-limited devices, Tiny Machine Learning (TinyML) has arisen to take intelligence to the extreme edge. Because of compression, pruning, quantization, and architecture-aware optimization, TinyML makes it possible to perform inference in real time. Frameworks like TensorFlow Lite Micro and CMSIS-NN (Common Microcontroller Software Interface Standard - Neural Network) have shown the feasibility of running lightweight ML models directly on microcontroller units (MCUs), which would reduce the latency and eliminate the need to transmit raw data to the central server.

Corresponding author: Prabal Verma (email: prabalverma357@gmail.com)

Federated Learning (FL) was first introduced by Google in 2017 to improve the text prediction in their mobile keyboard by distributing ML across multiple devices. FL has gained traction as a decentralized learning paradigm where client devices collaboratively train a global model without sharing raw data. Each client performs local updates using private data and communicates only parameter deltas, thus offering both privacy preservation and communication efficiency. FL is particularly well suited for IoT applications where data is sensitive and connectivity is sporadic. Yet, most existing FL frameworks assume resource-rich clients (e.g., smartphones or edge servers), rendering them infeasible for MCUs. Moreover, personalization under non-IID (Non-Independent and Identically Distributed) data distributions and frequent device dropouts adds further complexity to the FL.

In parallel, the development of sixth-generation (6G) wireless networks promises to address many of these systemic challenges. With features like Ultra-Reliable Low-Latency Communication (URLLC), massive Machine-Type Communication (mMTC), and AI-native orchestration, 6G infrastructure is expected to support distributed edge learning at scale, particularly in settings involving billions of heterogeneous devices [2]. Yet, current FL and TinyML frameworks are not designed to leverage the programmability and reliability offered by 6G, nor do they provide a unified mechanism for scalable, communication-efficient model training on constrained hardware.

A. Problem Statement

To address these challenges, this article proposes **TinyFed6G**, an inategrated framework that combines the power of TinyML, FL, and 6G for collaborative, communication-efficient, and personalized on-device training. In contrast to former static strategies assuming TinyML to be only on the inference side and ignoring heterogeneous

hardware, our approach dynamically balance the trade-off between quantization loss and communication overhead under the constraints of computation and energy and would still keep the accuracy and the fidelity to the real-world data variance.

B. Our Contributions

The main contributions of this work are:

- We introduce a dual-mode execution framework that allows resource-constrained TinyML devices to alternate between federated training and local inference modes. The mode switching is governed by real-time device conditions such as residual battery energy, memory pressure, and application-specific inference demands, enabling devices to contribute to global learning when feasible and serve real-time predictions during non-training intervals.
- We develop a profile-driven model selection mechanism in which each participating device is assigned an appropriate quantized model variant based on the computational and communication capabilities of each device.
- We design a communication- and memory-efficient FL pipeline combining lightweight training, last-layer personalization, and FedAvg-based aggregation to support non-IID adaptation without increasing uplink overhead. Additionally, semantic-aware compression filters updates using cosine similarity thresholds is employed to reduce redundant transmissions while preserving convergence and accuracy.
- We evaluate the performance of TinyFed6G and extensive simulation results under realistic TinyML constraints demonstrate that TinyFed6G achieves up to 6.5% personalization gain, 72% lower energy consumption, and 87% reduction in communication cost.

II. Related Work

The convergence of on-device ML, distributed learning, and next-generation wireless networks promises to unlock transformative applications. While research in each area has progressed significantly, major research gaps are still found in previous studies.

TinyML refers to the deployment of ML models on MCUs, moving the intelligence from the cloud to the extreme edge. To fit neural networks onto these constrained platforms, a suite of model compression techniques has been developed and refined. Quantization is the most fundamental technique, reducing the precision of model weights and activations from 32-bit floating-point to 8-bit or 4-bit integers. This yields up to a 4 times reduction in model size [3]. Similarly, pruning removes redundant or less impactful weights and filters, resulting in sparser architectures suitable for execution under tight memory constraints [3]. One of the most widely used deployment stacks for TinyML is TensorFlow Lite for Microcontrollers (TFLM), an embedded-friendly runtime for deploying pre-trained models on platforms such as ARM Cortex-M, ESP32, and RISC-V processors. Similarly, CMSIS-NN, a library optimized for ARM Cortex-M devices, enables acceleration of common neural operations using fixed-point arithmetic to reduce compute and memory usage [4]. Despite these advances, most TinyML implementations remain inference-only, where models are trained centrally and deployed to devices in static form.

FL emerged as a solution for training models on decentralized data. The seminal FedAvg algorithm introduced by McMahan et al. [5] laid the foundation for efficient parameter aggregation by averaging locally trained models. However, applying FL to ultra-constrained devices such as MCUs introduces a new dimension of challenges not encountered in mobile or cloud-edge settings. Notable experimental frameworks such as TinyFedTL [8] have demonstrated proof-of-concept FL systems running on devices like Arduino Nano BLE, ESP32, and Nordic nRF52840. Beyond FedAvg-based aggregation, several communication-efficient solutions have been proposed to reduce the uplink and downlink overhead [10]. A single global model often fails to perform well for all clients, especially when local data distributions are highly skewed (non-IID). Personalized FL (pFL) aims to create customized models for each device. [11] noted that a global model can be inferior for up to 40% of clients in realistic non-IID settings. Methods like FedPer [6] and FedRep [7] train a global model but allow each client to fine-tune the final layers locally to adapt to its specific data. Another study, named HCFL, groups clients with similar data distributions into clusters and trains a separate model for each cluster [9]. In addition to these, several optimization and personalization FL methods have emerged, such as FedProx, SCAFFOLD, and pFedMe for convergence stabilization under non-IID settings and Ditto, Per-FedAvg, and FedBN for improved personalization and domain generalization. Table I shows the comparative analysis of various previous work on FL for TinyML devices. The existing work addresses personalization or transfer learning for TinyML but focuses primarily on model-level or communication-level optimization in isolation. They have yet to be systematically integrated into 6G-style edge ecosystems that demand ultralow latency, seamless device orchestration, and personalized service delivery.

The 6G wireless communication paradigm is envisioned not only as a high-speed networking technology but also as a foundational enabler of intelligent distributed computation at the network edge. Unlike previous generations that primarily served human-centric applications, the integration of edge computing with Artificial Intelligence (AI)/ Machine Learning (ML) workloads is a core design target of 6G [12].As a result, FL frameworks are expected to become increasingly relevant in 6G environments. Recently, research integrating FL and 6G has moved from conceptual surveys [13] to architectural blueprints and early tests [14]. However, the integration of 6G into FL frameworks for TinyML still remains underexplored. The proposed TinyFed6G framework introduces a cross-layered framework for developing, deploying, and orchestrating FL applications on TinyML devices over 6G networks. Specifically, TinyFed6G will bridge the following identified research gaps:

- Most existing TinyML systems are inference-only, lacking on-device training support due to memory, energy,

TABLE I
COMPARATIVE ANALYSIS OF FEDERATED LEARNING FRAMEWORKS RELEVANT TO TINYML ENVIRONMENTS.

Technique	FL Strategy	Personalization	Communication Optimization	Key Outcome	Limitations
FedAvg [5]	Global aggregation of full model parameters	Shared global model across all clients	Full model updates each round	Foundational FL method; simple to implement	Fails under non-IID; high comm. and compute cost
FedPer [6]	Shared backbone with locally trained classifier head	Last-layer personalization	Partial model transmission	Accuracy approximately 81% and uplink reduction 60%	Static layer division; limited adaptability to device heterogeneity
FedRep [7]	Shared representation with client-specific classifier	Local classifier optimization	Reduced model updates	Accuracy approximately 83% and uplink reduction 70%	Additional local training time; lacks dynamic resource adaptation
TinyFedTL [8]	Transfer-learning-based FL using pre-trained backbone	Head-only adaptation	Partial model updates with quantization	Accuracy approximately 84% and communication reduction 90%	Depends on pre-trained model;; poor robustness under noise
HCFL [9]	Hierarchical cluster-based local aggregation	Cluster-specific model refinement	Intra-cluster aggregation reduces inter-edge traffic	Accuracy approximately 86% and communication reduction 75%	Requires cluster formation; added coordination cost
TinyFed6G (Proposed)	Profile-aware dual-mode FL with semantic compression	Last-layer fine-tuning	Quantization exchange with similarity-based suppression	86.2% accuracy, 6.5% personalization gain, 87% less comm., 72% energy saved	Profiling overhead; requires edge-tier coordination

and hardware limitations.

- Personalization strategies in FL remain too complex or memory intensive for real-world deployment in MCUs.

III. TINYFED6G: PROPOSED FRAMEWORK

Implementation of collaborative learning at wireless edge networks is challenging when applying traditional FL directly to MCUs. Moreover, the diverse computational and communication capabilities of TinyML-enabled devices further complicate model deployment and aggregation strategies. To address these challenges, we propose TinyFed6G, a hierarchical, communication-efficient, and personalized federated learning framework tailored for deployment in 6G-enabled edge environments. This section describes the architectural components, operational flow, and design principles of TinyFed6G framework.

A. System Overview

The proposed TinyFed6G framework adopts a three-tier federated learning hierarchy consisting of TinyML edge clients, 6G-enabled edge servers, and a centralized cloud server. The bottom layer comprises battery-powered edge devices equipped with ultra-low-power microcontrollers with stringent memory, computation, and energy constraints. Each device collects sensor data and runs a quantized ML model locally. Devices participate in the FL process opportunistically, based on their current energy state and memory availability. Training is initiated only when the resource budget satisfies system-defined thresholds, while inference remains the default mode. Each device is characterized by a profile vector given as $P_i = \{E_i, M_i, C_i\}$, where E_i denotes available energy, M_i available memory, and C_i communication bandwidth. The intermediate layer is composed of 6G base stations or mobile edge computing (MEC) units that serve as local federated coordinators. These FL coordinators are responsible for collecting model updates from clients, aggregating the local models using FedAvg, and scheduling model dispatch based on client profile vectors. The top layer includes a cloud-based controller that stores long-term model data, historical device behavior, and learning policies.

Besides tier architecture, TinyFed6G has several core design principles. In Fig. 1, the overall system design of the TinyFed6G framework is represented. The framework consists of three layers: a TinyML device layer for local sensing and switching between training and inference, an edge layer with 6G capabilities for model coordination and aggregation, and a cloud layer for long-term orchestration. The numbered annotations (1–9) illustrate the end-to-end process of training and inference, demonstrating how the framework enables heterogeneity, communicative efficiency, and personalization across edge devices in resource-constrained environments. The subsequent subsections will address each of the earlier operation components in detail.

B. Device Profiling and Adaptive Model Assignment

TinyFed6G introduces a device profiling mechanism that enables the 6G edge server to assign model variants to clients in a resource and context-aware manner. Each TinyML client device D_i is profiled using a three-dimensional resource vector such as $P_i = \{E_i, M_i, C_i\}$. These parameters are monitored locally and updated at the end of each FL round, or when a major change is detected. Each edge server has a set of pre-trained TinyML-compatible model variants, each differing in size, complexity, and accuracy, obtained using compression techniques such as quantization. Given a TinyML client device profile, the edge server utilizes a model assignment function that aims to maximize local inference quality while adhering to the resource budget. The model assignment function can be used to achieve various FL goals, such as maximizing accuracy or minimizing energy.

C. Dual-Mode Execution

In resource-constrained environments, forcing TinyML devices to participate in every FL round may lead to premature energy exhaustion, memory overflows, and degraded responsiveness. To mitigate these issues, TinyFed6G introduces a dual-mode execution mechanism, allowing each client device to dynamically alternate between local training and inference based on real-time profiling of its system state. In training mode, the TinyML client device engages in on-device model updates using local data. This involves executing low-footprint optimizers such as quantized SGD and transmitting compressed model deltas to the edge server. In contrast, in inference mode, the edge client device uses the currently assigned model to perform real-time predictions on incoming sensor data without engaging in training computations. The

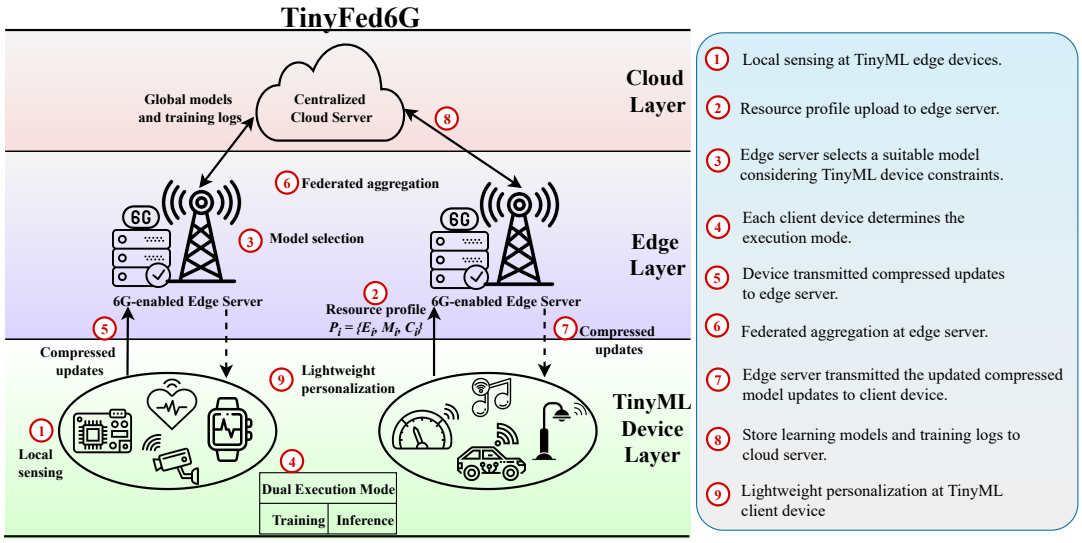


Fig. 1. System architecture of the proposed TinyFed6G framework. The architecture consists of three tiers: (i) TinyML-enabled edge devices that perform on-device sensing and participate in federated training based on local resource availability, (ii) 6G-enabled edge servers that coordinate edge clients, model selection, and FedAvg-based aggregation, and (iii) a centralized cloud controller that stores long-term learning logs and model states. Key operations include dual-mode execution, semantic compression, and lightweight personalization through fine-tuning at the last layer.

mode selection is governed by a threshold-based decision function at each client where available resources (energy, memory, and computation throughput) are compared with the respective threshold values. These thresholds are tunable based on application demands and network policies. Additionally, the decision function is evaluated periodically, that is, before each FL round or upon significant changes in resources. Additionally, energy model considers the compute cost of one local training epoch and the communication cost of transmitting the compressed model update.

D. Federated Aggregation

To support hierarchical collaboration across heterogeneous TinyML devices, TinyFed6G employs FedAvg-based aggregation at the 6G edge layer.

Instead of immediately transmitting model updates to the cloud, participating clients forward their compressed updates to the nearest 6G base station or edge server. The edge coordinator consolidates the received deltas using the conventional FedAvg [5] algorithm, calculated as:

$$w^{t+1} = \sum_{i=1}^{N} \frac{n_i}{n} w_i^t \tag{1}$$

where w_i^t denotes the model weights from device D_i, n_i the number of local samples, and $n = \sum_i n_i$ the global batch size. The resulting global model is then disseminated to eligible clients based on their P_i profile vectors. This localized aggregation reduces round-trip latency, enhances responsiveness, and limits backhaul traffic to the cloud. Additionally, cloud servers periodically synchronize with edge servers to refresh global learning trends, ensuring long-term consistency.

The choice of the classical FedAvg algorithm is result of its simplicity and computational efficiency. While advanced aggregation schemes such as FedProx, SCAFFOLD, or FedNova can offer improved convergence on highly non-IID data [13], but the underlying computation overhead and increase in memory requirement are challenging to sustain on an MCU with less than 256 KB of RAM. The proposed framework is also compatible with privacy-preserving and secure aggregation mechanisms such as differential privacy and homomorphic encryption, which can be incorporated at the edge-server tier.

E. Semantic Compression and Intent-Based Communication

In conventional FL, communication cost is dominated by gradient or weight updates often in the form of full-precision vectors. To further minimize uplink and downlink payloads, TinyFed6G integrates semantic-aware compression using quantization and intent-based update filtering. Before transmitting model updates, clients compress the head-layer weights using 8-bit quantization, reducing communication volume without significant accuracy degradation. Moreover, the system adopts intent-based networking, wherein only "*meaningful*" or semantically relevant updates are transmitted. This is determined by comparing the cosine distance between local and previous global model snapshots. If the change is minor, updates are locally retained to save bandwidth. The 6G infrastructure supports this paradigm through native features such as semantic radio layers.

F. Lightweight Personalization on TinyML Clients

In FL, especially those involving non-IID data across heterogeneous clients, the global model learned through aggregation may fail to generalize well to each local distribution. This is particularly problematic in TinyML environments where sensor data patterns are highly device-specific, yet constraints on memory and compute severely limit the feasibility of conventional personalization strategies. TinyFed6G addresses

this challenge by enabling lightweight personalization at the client side. Let d_i denote the local dataset of the TinyML edge client, D_i which follows a potentially unique data distribution. The global model w^t learned via FedAvg may not minimize the local loss, resulting in degraded inference accuracy. Personalized learning aims to find a locally adapted model $\hat{w}^i$, computed as:

$$\hat{w}^i = \arg\min_{w} L_i(w), \quad \text{s.t. } \|w - w^t\|_2 \leq \delta \tag{2}$$

Where δ represents the allowed deviation from the global model, constrained by the client's available memory and compute budget. Additionally, each edge client fine-tunes only the final (classification) layer of its received model using local data while freezing the backbone layers. This approach minimizes memory usage and preserves global feature consistency across clients.

The methods for adaptive usage allocation and profiling in TinyFed6G are computationally lightweight, even for large deployments. Each client sends a short profile vector with three elements, which the edge server evaluates against a small set of pre-quantized model types to determine a suitable assignment. This evaluation can be done in $O(N.M)$ where N is the number of active clients and M is the number of models. Since M is constant, the allocation cost increases linearly with the number of clients i.e. $O(N)$.

IV. Experimentation and Discussion

To validate the effectiveness and feasibility of the proposed framework in realistic edge-driven 6G environments, a series of simulation experiments are conducted on a standard dataset, and results are compared with state-of-the-art benchmark FL algorithms. The evaluation focuses on key performance indicators such as model accuracy, communication cost, convergence behavior and energy efficiency under non-IID data distributions. This section further discusses the simulation environment, dataset configuration, benchmark methods, evaluation metrics, and obtained results during experimentation.

The proposed framework is simulated using a layered hardware-software stack that emulates the computational and communication characteristics of a realistic 6G edge environment. Experimentation is carried out on NS-3 coupled with Python. Each TinyML client is modeled after ultra-low-power microcontroller units such as the ARM Cortex-M4 and RISC-V-based AI chips, with a memory constraint of 256 KB RAM and 1 MB flash. These clients operate under real-world power constraints and support INT8-quantized Convolutional Neural Network (CNN) models using TensorFlow Lite. Device heterogeneity is introduced by varying energy budgets, memory capacity, and bandwidth profiles. Local training employed the Adam optimizer (learning rate = 0.001, batch size = 32) for up to three epochs per communication round, with 20% of clients randomly sampled each round to reflect partial participation. The intermediate layer comprises virtualized 6G-enabled MEC servers with 8-bit head-layer updates using FedAvg and communication over a 0.5–10 Mbps channel. Each edge server is associated with 5-10 client devices. The communication The profile vector is updated at the end of each communication round or after a defined threshold, whichever occurs first. The energy model is configured using the MCU's datasheet report.

To evaluate the proposed TinyFed6G framework under realistic TinyML and federated learning constraints, we use the Google Speech Commands dataset [15] available publicly on Kaggle. This dataset contains approximately 64,727 audio files of 30 short words (e.g., "yes", "no", "stop", "go"), sampled at 16 kHz and spoken by thousands of different users. We restrict our evaluation to the standard 12-command subset, giving us approximately 35,000 labeled examples. Each audio clip is transformed into a 32×32 Mel-Frequency Cepstral Coefficients (MFCC) spectrogram, allowing efficient input representation for lightweight CNN-based TinyML models. To mimic practical edge scenarios involving non-IID data distributions, the dataset is partitioned across 50 simulated TinyML clients using the label skew strategy with a concentration parameter $\alpha = 0.3$, such that each client observes samples from only 2–3 dominant labels. We categorize these 50 clients into 10 federated edge servers, with each server integrating updates from 5 clients. For experimental instances, each client utilizes 70% of local data for training, 10% for validation, and 20% for testing and evaluation. The test set is non-overlapping among clients in order to prevent data leakage.

In order to thoroughly assess the efficacy of the TinyFed6G framework, we compared it with various state-of-the-art FL methods. The selection of these baseline techniques was made carefully considering their close relevance with the proposed research.

- FedAvg [5]: This method can be viewed as a basic FL method in which specific client models are developed with the help of private data and then the global model is formed by averaging the models of the clients periodically through a central server.
- FedPer [6]: FedPer splits the model into shared and personalized layers, typically fixing the backbone and allowing clients to train only the classifier head. It aligns closely with TinyFed6G's lightweight personalization strategy but does not incorporate device profiling or dual-mode execution.
- TinyFedTL [8]: This method explores TinyML oriented transfer learning using a pre-trained global model where clients perform on-device fine-tuning using their resource-limited configuration.
- FedRep [7]: FedRep fixes representation layers and allows clients to learn a personalized classifier. Particularly effective in non-IID and heterogeneous environments, this study represents a strong personalization-focused benchmark.

Each baseline is re-implemented under the same simulation framework, dataset and resource constraints. For fairness, all models share the same CNN backbone architecture (quantized, 3-layer TinyCNN).

To analyze convergence behavior and predictive performance, we evaluate model accuracy across 50 federated learning rounds for the proposed work and compare it with four baseline FL methods: FedAvg, FedPer, TinyFedTL, and FedRep. Fig. 2 presents the accuracy progression over 50

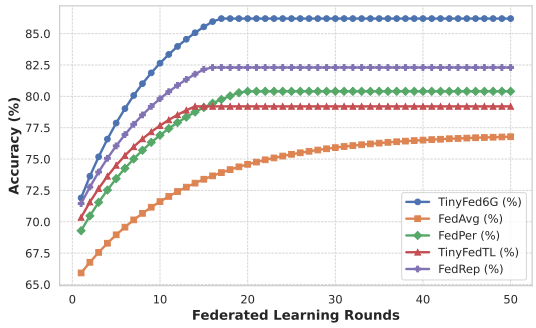


Fig. 2. Model Accuracy across Federated Learning Rounds

FL rounds. TinyFed6G demonstrates the fastest convergence, reaching 86.42% accuracy by round 17. Subsequent rounds up to 50 are included only to show long-term behavior and marginal improvement. In contrast, FedAvg exhibits the slowest convergence, achieving only 77.1% at round 50. FedPer improves performance to 80.4% by decoupling the classifier head but suffers due to static layer assignment. TinyFedTL initially rises rapidly but plateaus early (approximately 79.2%), indicating unstable fine-tuning under constrained conditions. FedRep, with fixed representation layers, performs better than FedPer and TinyFedTL, reaching 82.3%, but still lags behind TinyFed6G. While 50 FL rounds are used here to analyze convergence, realistic TinyML deployments may constrain active training to 10–30 rounds. TinyFed6G addresses this by supporting adaptive client participation, compressed communication, and selective local updates, sustaining feasibility in resource-constrained scenarios.

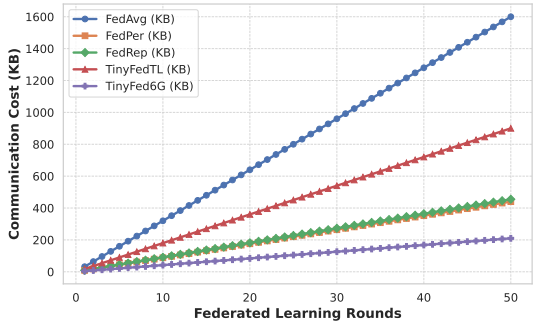


Fig. 3. Communication Cost across Federated Learning Rounds

Moreover, the communication overhead is quantified as the average amount of data transmitted per client for the federated learning round, including uplink and downlink transmissions. The communication overhead per FL round under different schemes is presented in Fig. 3. FedAvg has the highest communication cost as it needs to send the entire model in each round, up to 1.6 MB per client in 50 rounds. FedPer and FedRep lower this cost remarkably by sending updates only from the classifier layer, yielding 72–75% fewer communicated bits than FedAvg. The TinyFed6G model best reduces the communication footprint, which is attributed to its quantization and semantic compression scheme that filters out small updates through cosine similarity thresholds. TinyFed6G saves around 87% of communication over FedAvg and by around 52% against FedRep, over 50 rounds, which makes it a great choice for 6G-enabled TinyML networks, where bandwidth is a scarce resource.

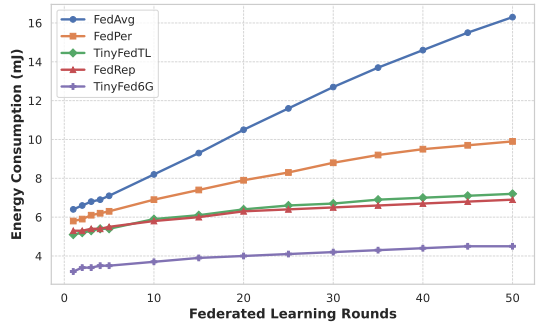


Fig. 4. Energy Consumption across Federated Learning Rounds

Next, we evaluate the estimate of the average energy expenditure at the client during each round, considering both training and inference modes. Fig. 4 illustrates the round-wise cumulative energy consumption per client for the proposed TinyFed6G and competing baselines. As the results reveal, FedAvg incurs the highest energy cost, reaching approximately 16.3 mJ by Round 50. FedPer reduces this burden by freezing feature layers and training only the classifier head, thus limiting compute requirements, but still lacks runtime adaptivity. TinyFedTL and FedRep show better energy performance due to partial local training and shared representation strategies, consuming approximately 7.2 mJ and approximately 6.9 mJ, respectively, by Round 50. In contrast, TinyFed6G demonstrates the lowest energy footprint, only 4.5 mJ by Round 50, giving an approximately 72% reduction compared to FedAvg.

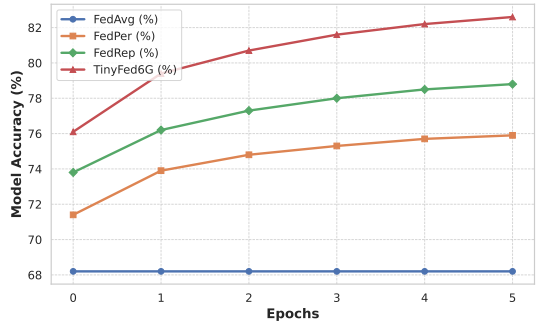


Fig. 5. Personalization Gain vs Epochs

The personalization gain metric evaluates how effectively each FL technique adapts to non-IID local data distributions through fine-tuning. As illustrated in Fig. 5 TinyFed6G clearly outperforms all the baseline methods in terms of accuracy uplift obtained with only few local adaptation epochs. FedAvg does not exhibit any personalization gain due to the fact it does not allow any client adaptation after aggregation. Thus, the test accuracy is the same across epochs. Through profile-aware assignment and last-layer fine-tuning, TinyFed6G obtains the

largest gain by around 6.5% in 2 epochs, and by around 6.5% in the next 3 epochs. This is critical for real time edge devices as low adaptation cost results in high performance gain. Note that our TinyFed6G can achieve more than 82% accuracy after 5 epochs of lightweight tuning.

To evaluate the TinyFed6G's behavior under 6G settings, latency and jitter are evaluated under variable link capacities such as 1, 5, and 10 Mbps. The baseline FedAvg model transmitted approximately 32 KB parameters per communication round. In contrast, TinyFed6G employed semantic filtering with a cosine similarity threshold of 0.15, reducing the average payload to around 4 KB per round giving a 87% compression. At 5 Mbps, TinyFed6G achieved an average round latency of 328 ms compared with 717 ms for FedAvg. At 1 Mbps, latency gain is even more prominent, with TinyFed6G round latency going from 1.13 s to 0.38 s, due to the compressed communication payloads. Jitter values followed a similar trend, decreasing from 11.2 ms in FedAvg to 4.7 ms in TinyFed6G at 5 Mbps, giving a 58% reduction. The experimental work has shown that TinyFed6G achieves substantial improvements over prior FL approaches.

V. Conclusion and Future Work

In this paper, we proposed TinyFed6G, a hierarchical and resource-aware federated learning framework designed specifically for TinyML devices operating within 6G-enabled edge networks. TinyFed6G resolves these key challenges of implementing collaborative learning in places with limited resources by employing personalized model allocation based on device profile, dual-mode execution for flexible participation, and semantic compression to minimize redundant communications. Simulation results on the Google Speech Commands dataset demonstrate that the proposed framework achieves 86.2% accuracy, a 6.5% personalization improvement, and reductions of 72% in energy consumption compared with state-of-the-art baseline methods. These results validate the framework's potential for real-world deployment, such as voice-controlled embedded systems and intelligent IoT endpoints. Despite these advantages, the current evaluation is based on a simulation environment, rather than directly measuring MCUs performance. Future work will therefore focus on deploying TinyFed6G on real TinyML hardware platforms to evaluate energy and end-to-end latency. Additionally, the work can be extended by integrating lightweight secure aggregation and differential-privacy modules to protect client updates.

Software Availability

The implementation code supporting the findings of this study is publicly available for researchers at https://github.com/jvibha0/TinyFed6G.

References

[1] J. S. Yalli, M. H. Hasan, and A. A. Badawi, "Internet of things (iot): origins, embedded technologies, smart applications, and its growth in the last decade," *IEEE access*, vol. 12, pp. 91 357–91 382, 2024.

[2] Z. Yang, M. Chen, K.-K. Wong, H. V. Poor, and S. Cui, "Federated learning for 6g: Applications, challenges, and opportunities," *Engineering*, vol. 8, pp. 33–41, 2022.

[3] S. Heydari and Q. H. Mahmoud, "Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions," *Sensors*, vol. 25, no. 10, p. 3191, 2025.

[4] A. Maciá-Lillo, S. Barrachina, G. Fabregat, and M. F. Dolz, "Optimizing convolutions for deep learning inference on arm cortex-m processors," *IEEE Internet of Things Journal*, vol. 11, no. 15, pp. 26 203–26 219, 2024.

[5] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.

[6] A. Z. Tan, H. Yu, L. Cui, and Q. Yang, "Towards personalized federated learning," *IEEE transactions on neural networks and learning systems*, vol. 34, no. 12, pp. 9587–9603, 2022.

[7] L. Collins, H. Hassani, A. Mokhtari, and S. Shakkottai, "Exploiting shared representations for personalized federated learning," in *International conference on machine learning*. PMLR, 2021, pp. 2089–2099.

[8] K. Kopparapu, E. Lin, J. G. Breslin, and B. Sudharsan, "Tinyfedtl: Federated transfer learning on ubiquitous tiny iot devices," in *2022 IEEE international conference on pervasive computing and communications workshops and other affiliated events (PerCom Workshops)*. IEEE, 2022, pp. 79–81.

[9] Y. Guo, X. Tang, and T. Lin, "Enhancing clustered federated learning: Integration of strategies and improved methodologies," in *The Thirteenth International Conference on Learning Representations*, 2025.

[10] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-iid data," *IEEE transactions on neural networks and learning systems*, vol. 31, no. 9, pp. 3400–3413, 2019.

[11] V. Kulkarni, M. Kulkarni, and A. Pant, "Survey of personalization techniques for federated learning," in *2020 fourth world conference on smart trends in systems, security and sustainability (WorldS4)*. IEEE, 2020, pp. 794–797.

[12] M. A. Hassan and F. Granelli, "Deep learning-driven optimal beam prediction for drone connectivity via flying-ris and base station," in *2025 IEEE 101st Vehicular Technology Conference (VTC2025-Spring)*. IEEE, 2025, pp. 1–6.

[13] S. Jere, Y. Song, Y. Yi, and L. Liu, "Distributed learning meets 6g: A communication and computing perspective," *IEEE Wireless Communications*, vol. 30, no. 1, pp. 112–117, 2023.

[14] F. H. Alghamedy, N. El-Haggar, A. Alsumayt, Z. Alfawaer, M. Alshammari, L. Amouri, S. S. Aljameel, and S. Albassam, "Unlocking a promising future: integrating blockchain technology and fl-iot in the journey to 6g," *IEEE Access*, 2024.

[15] P. Warden, "Speech commands: A public dataset for single-word speech recognition," https://www.tensorflow.org/datasets/catalog/speech_commands, 2017, accessed: July 2025.

Biographies

Vibha Jain is working as an Assistant Professor in the Department of Computer Science and Engineering at Thapar Institute of Engineering & Technology, Patiala, Punjab, India. She completed her Ph.D. in Computer Science and Engineering from the Netaji Subhas University of Technology, Delhi. Her research area includes the Internet of Things, Blockchain, Edge/fog Computing, and Resource Scheduling/allocation.

Aditya Gupta has been serving as an Assistant Professor with Thapar Institute of Engineering and Technology, Patiala (Punjab), India. He received his doctoral degree in Computer Science and Engineering from Dr B R Ambedkar National Institute of Technology, Jalandhar. His research interests include Healthcare Informatics, Predictive Analytics, and Cyber-Physical Systems.

Prabal Verma , currently an Assistant Professor in the Department of Information Technology at NIT Srinagar, India, holds a Ph.D. in Computer Science and Engineering from Guru Nanak Dev University, Amritsar, and has research interests in Big Data Analytics, Machine/Deep Learning, and Edge–Cloud Computing.

Sukhpal Singh Gill [SFHEA] is an Assistant Professor in Cloud Computing at the School of Electronic Engineering and Computer Science, Queen Mary University of London, UK. Prior to this, he was postdoc at Lancaster University, UK and CLOUDS Lab, University of Melbourne, Australia. Dr. Gill is serving as an Editor-in-Chief for IGI Global IJAEC and Area Editor for Springer Cluster Computing Journal, also serving as an Associate Editor in IEEE IoT, Nature Scientific Reports, Wiley SPE, Elsevier IoT, Wiley ETT, Elsevier Telematics and Informatics Reports and IET Networks Journals.